[COLLEGE / UNIVERSITY NAME]

DEPARTMENT OF COMPUTER APPLICATIONS

# MCA MINI PROJECT REPORT

# Fitness Habit Analysis and Personalized Workout Recommendation System for Gym Users

A web-based fitness habit analysis and personalised workout recommendation system combining a transparent rule engine with a hybrid machine-learning model

**Submitted by: [STUDENT NAME]**

**Submitted to: [GUIDE NAME]**

**Roll number: [ROLL NUMBER]**

**Course: Master of Computer Applications (MCA)**

Academic year: [ACADEMIC YEAR]

# ABSTRACT

This mini project presents a Fitness Habit Analysis and Personalized Workout Recommendation System for Gym Users, implemented as a dependency-free, single-page web application with an optional embedded machine-learning layer. The front end collects a fitness profile through a structured form and runs a transparent rule-based analysis that scores six habit dimensions, detects problems with 26 rules, generates a personalised workout plan and a seven-day schedule, and renders progress charts together with lifestyle advice. On top of the rule engine, a Python service trains a Random Forest classifier that predicts the fitness habit category of a gym user and a K nearest neighbours recommender that ranks suitable workout categories, and exposes both through a small REST API that feeds a Hybrid Fitness Intelligence panel on the dashboard.

The machine-learning stage was executed on a labelled cohort of 1,733 rows built from the public Gym Members Exercise Dataset (973 real sessions) together with a documented, seeded simulation of 760 rows, of which 863 cells were deliberately left blank so that the imputation path is always exercised. The Random Forest achieved a held-out test accuracy of 0.8876 (macro F1 0.8277) and the recommender reached a test accuracy of 0.8588 with top-two and top-three hit rates of 0.9481 and 0.9683. The implementation is covered by an automated test suite, a rule-engine sweep over 960 profile combinations and a full browser integration test. The system runs entirely on the client except for the optional prediction API, keeps all user data in the browser, and is deployed as a serverless Flask application on Vercel.

The report describes the problem identification and statement, the objectives, the existing and proposed systems, the scope, the requirements, the system architecture, the dataset and its preprocessing, the actual machine-learning algorithms and workflow, the implementation, the results and analysis, the testing, the limitations, the future scope, the conclusion and the references.

# TABLE OF CONTENTS

**Chapter 1: Introduction**
**.....7**
1.1. Background
.....7
1.1.1 Background of the project
.....7
1.2. Motivation
.....7
1.3. Organisation of the report
.....7
**Chapter 2: Problem Identification**
**.....9**
2.1. Context of the problem
.....9
2.2. Problems identified by the system
.....9
2.3. Problems identified outside the rule engine
.....9
**Chapter 3: Problem Statement**
**.....11**
3.1. Problem statement
.....11
3.2. Constraints of the problem
.....11
**Chapter 4: Objectives**
**.....12**
4.1. Primary objective
.....12
4.2. Secondary objectives
.....12
**Chapter 5: Existing System**
**.....13**
5.1. The system before the model
.....13
5.2. Working of the existing system
.....13
5.3. Drawbacks of the existing system
.....13
5.4. Summary
.....13
**Chapter 6: Proposed System**
**.....14**

6.1. Overview
.....14
6.2. Proposed workflow
.....14
6.3. Features of the proposed system
.....14
**Chapter 7: Scope**
**.....15**
7.1. In scope
.....15
7.2. Out of scope
.....15
**Chapter 8: Software and Hardware Requirements**
**.....16**
8.1. Functional requirements
.....16
8.2. Non-functional requirements
.....16
8.3. Software requirements
.....16
8.4. Hardware requirements
.....17
**Chapter 9: System Architecture**
**.....18**
9.1. Layered architecture
.....18
9.2. Module structure
.....18
9.3. Data flow
.....19
9.4. Deployment architecture
.....20
**Chapter 10: Dataset Description**
**.....21**
10.1. Source of the data
.....21
10.2. Class distribution
.....21
10.3. Data dictionary
.....22
10.4. Missing values
.....22
10.5. Label generation rules
.....23

**Chapter 11: Data Preprocessing**
**.....24**
11.1. Validation before training
.....24
11.2. Imputation
.....24
11.3. Encoding of categorical features
.....24
11.4. Scaling
.....24
11.5. Train and test split
.....24
**Chapter 12: Actual Machine Learning Algorithm and Model Workflow**
**.....26**
12.1. Choice of algorithms
.....26
12.2. Random Forest classifier
.....26
12.3. K nearest neighbours recommender
.....26
12.4. Model workflow
.....27
12.5. Inference for a user profile
.....27
**Chapter 13: Implementation Details**
**.....29**
13.1. Implementation of the front end
.....29
13.2. Implementation of the machine-learning service
.....29
13.3. The application programming interface
.....30
13.4. Protection of the deployed files
.....30
13.5. Screenshots of the running application
.....30
**Chapter 14: Results and Analysis**
**.....34**
14.1. Evaluation methodology
.....34
14.2. Results of the Random Forest classifier
.....34
14.3. Results of the K nearest neighbours recommender
.....36

14.4. Comparison with the baseline
.....38
14.5. Result rendered in the application
.....38
14.6. Summary of the results
.....39
**Chapter 15: Testing**
**.....40**
15.1. Strategy
.....40
15.2. Automated test suite of the machine-learning package
.....40
15.3. Testing of the rule engine
.....41
15.4. Testing of the interface
.....41
15.5. Testing of the deployed application
.....41
**Chapter 16: Limitations**
**.....42**
16.1. Limitations of the implementation
.....42
**Chapter 17: Future Scope**
**.....43**
17.1. Planned extensions
.....43
17.2. Extensions of the machine-learning layer
.....43
**Chapter 18: Conclusion**
**.....44**
18.1. Conclusion
.....44
**Chapter 19: References**
**.....45**
19.1. References
.....45
19.2. Project documents
.....45
19.3. Note on information that still has to be filled in
.....45

# CHAPTER 1
# INTRODUCTION

## 1.1. Background

Fitness training is one of the most common self-improvement activities, yet it is also one where adherence collapses quickly. Gym users typically start with enthusiasm, follow an irregular routine, skip sessions during busy weeks, and stop tracking their progress. Nutrition, hydration, sleep and daily activity quietly drift away from what the training goal requires, and the user rarely connects these habits to the stall in results.

A personal trainer solves this problem by asking structured questions, observing the client over weeks and adjusting the programme. Most gym members do not have that continuous supervision, and a coach cannot maintain a personal conversation with every member of a large club. The consequence is generic advice: the same plan is handed to beginners and advanced athletes, and no one explains why a particular habit is holding the trainee back.

### *1.1.1 Background of the project*

This project was developed as a college field project around a single user-facing web application. The user fills in a structured fitness profile; the application scores the habits, identifies problems, generates a personalised workout plan and weekly schedule, tracks daily progress with charts, and offers lifestyle guidance. A machine-learning layer, called the Hybrid Fitness Intelligence Model, was later added on top of the original rule-based engine so that the habit profile of a trainee could also be classified by a trained model and a workout category could be recommended from a labelled cohort of similar trainees. This report documents that complete implementation.

## 1.2. Motivation

The motivation of the work is threefold. First, the analysis must be transparent: every score shown to the user is produced by a rule that is displayed in the application, so the result can be trusted and explained. Second, the recommendation must be personal: the workout category depends on the goal, the training frequency, the experience level, the session length, the equipment and the stated training preference. Third, the whole system must work without infrastructure: the front end has no framework, no build step and no external libraries, and the data of the user never leaves the browser.

## 1.3. Organisation of the report

The report is organised as follows. Chapter 2 and Chapter 3 identify the problem and state it formally. Chapter 4 lists the objectives, Chapter 5 describes the existing system and Chapter 6 the proposed system. Chapter 7 gives the scope and Chapter 8 the software and hardware requirements.

Chapter 9 presents the system architecture, Chapter 10 the dataset, Chapter 11 the preprocessing and Chapter 12 the actual machine-learning algorithms and workflow. Chapter 13 covers the implementation, Chapter 14 the results and analysis, Chapter 15 the testing, and Chapters 16 to 19 the limitations, future scope, conclusion and references.

# CHAPTER 2
# PROBLEM IDENTIFICATION

## 2.1. Context of the problem

The problem area is the daily practice of gym training together with the lifestyle habits that support it. The behaviour is measured through six habit dimensions, each scored from zero to one hundred against the targets of the user's own profile. The weights of these dimensions are fixed in the rule engine so that the overall habit score is a weighted sum of the six dimensions.

*Table 2.1 Habit dimensions and their weights*

| Habit dimension | Weight | What is measured |
|---|---|---|
| Workout consistency | 25% | Completed sessions compared with the sessions planned for the week |
| Diet and nutrition | 20% | Diet self-rating, protein meal coverage and junk food frequency |
| Hydration | 15% | Glasses of water compared with the target derived from body weight |
| Sleep and recovery | 15% | Average sleep against the 7 to 9 hour range and screen use before bed |
| Daily activity | 15% | Average daily steps against the step target for the goal |
| Motivation and tracking | 10% | Motivation rating and the habit of recording progress |

## 2.2. Problems identified by the system

The rule engine implements 26 problem-detection rules. Each rule carries a severity and an impact weight, and the detected problems are sorted by impact so that the most damaging behaviour is shown first. The rules are grouped below by the dimension they belong to; the severity distribution of the complete rule set is nine high-severity rules, thirteen medium-severity rules and four low-severity rules.

*Table 2.2 Problem rules grouped by habit dimension*

| Dimension | Representative rules detected by the engine |
|---|---|
| Consistency | No structured training, very low training frequency, plan gap between planned and actual sessions, three or mo |
| Nutrition | Low diet quality, high or moderate junk food frequency, low protein coverage, fat-loss goal without resistance |
| Hydration | Dehydration, water intake below the derived target |
| Sleep | Sleep deficit, sleep below the optimal range, screen use before bed, sleep limiting endurance gains |
| Activity | Sedentary days, steps below the target |
| Motivation | Low motivation, no progress tracking habit |
| Availability | Availability mismatch between equipment and plan, long sessions for a beginner, session time mismatch |
| Body composition | Muscle-gain goal at a high BMI, fat-loss goal at a low BMI |

## 2.3. Problems identified outside the rule engine

The rules alone cannot express learned interactions between the attributes of a trainee. A rule reads one condition at a time, whereas the habit of a user emerges from the combination of frequency, recovery, nutrition and activity. In addition, a rule produces a yes-or-no answer and cannot express how confident the system is about a classification, nor can it rank several workout options by their similarity to previously observed trainees. These two gaps - learned interaction and graded confidence - are the problems addressed by the machine-learning layer reported in Chapters 10 to 14.

# CHAPTER 3
# PROBLEM STATEMENT

## 3.1. Problem statement

Gym users manage their training with irregular habits and no supervision. For such a user, the existing application can score habits and list problems through fixed rules, but it cannot determine which fitness habit class the user belongs to, it cannot express any confidence in that judgement, and it cannot recommend a workout category that resembles what similar trainees actually followed.

The problem may therefore be stated as follows: a supervised learning approach is required that takes the fitness profile of a gym user, predicts the habit class of that user with a measurable accuracy, ranks the suitable workout categories by their similarity to a labelled cohort, and returns the result to the same dashboard together with the evaluation of the model that produced it, while the service must also report honestly when the training data is not sufficient to support a prediction.

## 3.2. Constraints of the problem

- The profile is captured in the browser and must be translated into model features without a database.
- The labelled cohort must be built from a public source, and every synthetic part of it must be documented.
- Predictions must come from a fitted model; no answer may be hard-coded in the interface.
- A deployment may have a read-only file system, so the model must be able to fit itself in memory.

# CHAPTER 4
# OBJECTIVES

## 4.1. Primary objective

The primary objective is to extend the existing rule-based fitness application with a hybrid machine-learning model that classifies the fitness habit profile of a gym user and recommends workout categories, and to present the result, its confidence and its evaluation on the existing dashboard.

## 4.2. Secondary objectives

• To build and document a labelled training cohort from a public gym dataset.

• To derive both target labels from transparent rules and to inject controlled label noise so that the task is not perfectly separable.

• To encode the preprocessing as scikit-learn pipelines so that training and inference can never disagree.

• To evaluate both models with a stratified split, five-fold cross-validation, per-class metrics, confusion matrices, feature importances and majority-class baselines.

• To expose the models through a small JSON API with health, metrics, predict and train endpoints.

• To implement graceful behaviour for offline use and for insufficient training data.

• To protect source files and model artefacts from being served over HTTP.

• To deploy the complete system on Vercel with a read-only model strategy.

• To validate the result with automated tests for the dataset, the labels, the preprocessing, the API and the deployed entrypoint.

# CHAPTER 5
# EXISTING SYSTEM

## 5.1. The system before the model

The project already contained a complete front end before the machine-learning layer was added: a dashboard with an overall habit score gauge and a six-dimension breakdown, an analysis page with prioritised problems and ranked solutions, a personalised workout plan, a seven-day schedule, a progress page with charts and CSV or JSON export, and a page of lifestyle tips. All of it is written in plain HTML, CSS and JavaScript with no framework and no build step, and all user data is stored in the browser.

## 5.2. Working of the existing system

The existing analysis is entirely rule-based. The engine computes the six dimension scores, applies the 26 problem rules, sorts the detected problems by impact, derives the personal targets from the Mifflin-St Jeor basal metabolic rate, and generates the workout plan from a template library with equipment substitution and prescription tables. The plan generator chooses a training split for the goal, maps the session types onto the training days of the week, sizes the session to the available time and attaches progression, deload and recovery rules.

## 5.3. Drawbacks of the existing system

The rule engine is explainable but static. Table 5.1 compares what the existing system could and could not do before the hybrid model was introduced.

*Table 5.1 Comparison of the existing system with the proposed system*

| Aspect | Existing rule-only system | After the hybrid model |
|---|---|---|
| Habit classification | One rule set, no alternatives shown | Five trained classes with a probability for each class |
| Confidence | Not available; the score is deterministic | Reported as the probability of the winning class |
| Recommendation | Fixed mapping from goal to session type | Nine-neighbour vote with a ranked top three and match scores |
| Similarity evidence | None | The labels of the nearest training profiles are displayed |
| Evaluation | None; correctness could not be measured | Accuracy, macro and weighted F1, five-fold cross validation and |
| Behaviour on missing data | Assumed defaults silently | Explicit insufficient-data message instead of a prediction |

## 5.4. Summary

The existing system therefore remained in place as the rule-based backbone of the application; the machine-learning layer was integrated beside it so that the dashboard can show both readings of the same profile, and the concrete exercise prescriptions continue to come from the plan generator.

# CHAPTER 6
# PROPOSED SYSTEM

## 6.1. Overview

The proposed system adds a Hybrid Fitness Intelligence layer to the existing application. Two supervised models are trained on a labelled fitness cohort: a Random Forest classifier predicts the habit category of the trainee from the full habit profile, and a K nearest neighbours recommender ranks workout categories from the goal, the training frequency, the experience level, the session length, the equipment, the activity level and the stated training preference. Both models are wrapped in preprocessing pipelines and are served by a small Flask application that also serves the front end.

## 6.2. Proposed workflow

• The user completes the profile form in the browser.

• The rule engine produces the scores, problems, plan and schedule as before.

• The machine-learning client posts the same profile to the prediction endpoint.

• The service converts the profile into a feature frame, applies the fitted pipelines and returns the habit class with its class probabilities, the three ranked workout categories with match scores and the labels of the nearest training profiles.

• The dashboard panel renders the result together with the model name, the training row count and the test accuracy of both models.

## 6.3. Features of the proposed system

*Table 6.1 Features of the proposed system*

| Feature | Description |
|---|---|
| Habit classification | Five habit categories predicted by a Random Forest with a probability per class |
| Workout recommendation | Three ranked workout categories produced by a nine-neighbour vote |
| Transparency | Class probabilities, the accuracy of the model and the labels of the nearest profiles are shown to the use |
| Honest degradation | If the service is unreachable or the training data is insufficient, the panel reports the reason instead of in |
| Reproducibility | Fixed random seeds, a pinned dependency set and a committed evaluation report |
| Serverless operation | The model is loaded from a bundled artefact and is fitted in memory when the file system is read-only |

# CHAPTER 7

# SCOPE

## 7.1. In scope

• A browser-based application covering habit analysis, personalised plans, weekly schedules, progress tracking and lifestyle guidance.

• A supervised classifier for the habit category and a nearest-neighbour recommender for the workout category.

• A documented labelled dataset with explicit provenance for every column.

• Preprocessing, training, evaluation and artefact management as Python modules.

• A JSON API with health, metrics, predict and train endpoints, plus static serving of the front end.

• Automated tests for the data, the labels, the preprocessing, the models, the API and the deployment entrypoint.

• Deployment of the complete stack as a serverless application.

## 7.2. Out of scope

• Medical, clinical or dietary diagnosis; the output is educational guidance only.

• Measurement of physiological outcomes; the cohort describes patterns in its own rows.

• User accounts, authentication, a central database or data synchronisation between devices.

• Integration with wearables, gym equipment or external fitness platforms.

• Prescription of individual exercises by the model; concrete sessions are produced by the rule-based plan generator.

# CHAPTER 8

# SOFTWARE AND HARDWARE REQUIREMENTS

## 8.1. Functional requirements

*Table 8.1 Functional requirements*

| ID | Requirement |
|---|---|
| FR1 | Capture and validate a fitness profile (age, sex, height, weight, goal, experience, training frequency, session length, equipmen |
| FR2 | Score six habit dimensions and detect problems with 26 ranked rules |
| FR3 | Derive personal targets for calories, protein, water and steps |
| FR4 | Generate a personalised workout plan from 58 exercises and 24 session templates |
| FR5 | Generate a seven-day schedule with real rest days |
| FR6 | Record daily check-ins and render five progress charts with CSV or JSON export |
| FR7 | Serve 41 lifestyle tips with a rotating tip of the day |
| FR8 | Classify the habit profile and rank workout categories through the prediction API |
| FR9 | Report the training row count, the test accuracy and the model names inside the interface |
| FR10 | Show an explicit notice when the model service is offline or the training data is insufficient |

## 8.2. Non-functional requirements

*Table 8.2 Non-functional requirements*

| Category | Requirement |
|---|---|
| Portability | Runs in any modern browser from a static folder or from a URL; no build step |
| Performance | The front end reacts immediately; the first request of the API may load the model |
| Data privacy | User data stays in the browser storage; the API does not persist any profile |
| Security | Python source, the model artefact and the training CSV are not served over HTTP |
| Maintainability | Configuration, labels, preprocessing, training and inference are separate modules |
| Reproducibility | Fixed random seed 42, stratified splits and pinned dependency versions |

## 8.3. Software requirements

*Table 8.3 Software requirements*

| Component | Version or requirement |
|---|---|
| Operating system | Windows, macOS or Linux (developed on Windows) |
| Browser | Chrome, Edge, Firefox or Safari, a recent version |
| Python | 3.9 or newer; the reported metrics were produced on Python 3.13.9 |
| pandas | 2.3.3 |
| scikit-learn | 1.7.2 |
| numpy | 2.3.5 |

| | |
|---|---|
| joblib | 1.5.2 |
| Flask | 3.1.2 |
| pytest | 8.4.2 |
| Node.js | Optional, only for the zero-dependency static server |
| Version control | Git, with the repository hosted on GitHub |
| Hosting | Vercel, for the serverless Flask deployment |
| Database | None; the application uses browser storage and CSV files |

## 8.4. Hardware requirements

The system has no special hardware demands. The front end runs in an ordinary browser, and the training stage fits a dataset of 1,733 rows on a standard development machine. The hardware used for the reported runs is listed below; the reader should confirm these figures against the laboratory machine that is submitted with the project.

*Table 8.4 Hardware requirements*

| Item | Requirement / configuration used |
|---|---|
| Processor | [TO BE COMPLETED - processor of the development machine] |
| Memory (RAM) | [TO BE COMPLETED - for example 8 GB or 16 GB] |
| Storage | [TO BE COMPLETED - free disk space, for example 1 GB] |
| Display | Any display supported by the browser, resolution 1280 x 720 or higher |
| GPU | Not required; both models are trained on the CPU |
| Network | Required only to download the public dataset and to deploy the application |

# CHAPTER 9

# SYSTEM ARCHITECTURE

## 9.1. Layered architecture

The system is organised into four layers. The presentation layer contains the single-page shell and the design system; the client layer contains the controller, the rule engine, the content libraries, the charts, the storage wrapper and the machine-learning client; the service layer is the Flask application with its four API endpoints and the inference modules; and the data layer holds the labelled CSV, the fitted pipelines and the evaluation report. Figure 1 shows the four layers and the direction of the calls between them.

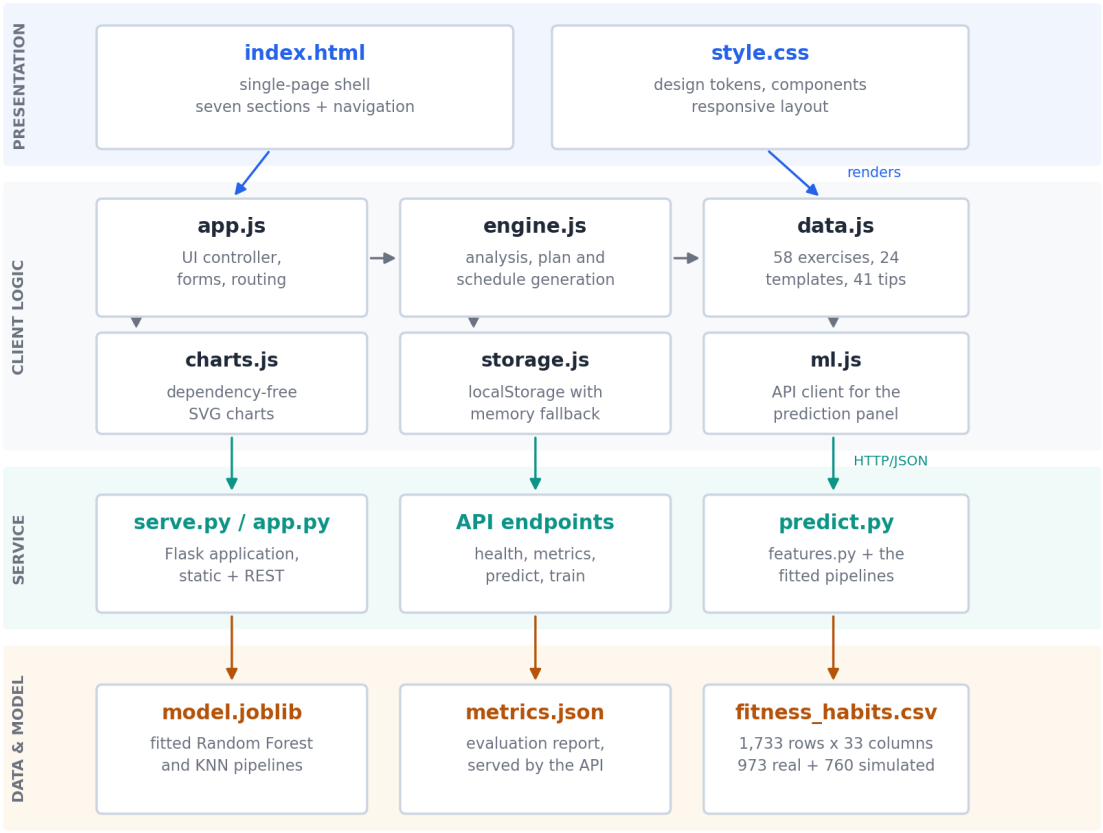



*Figure 1. Layered system architecture. The browser renders the presentation layer, the client layer performs the analysis and calls the service layer over HTTP, and the service layer uses the fitted pipelines and the labelled cohort of the data layer.*

## 9.2. Module structure

The application divides into eight functional modules, shown in Figure 2. The analysis, planning, tracking and guidance modules are served by the rule engine and the content library; the

persistence module wraps the browser storage; the presentation module owns the markup and the styles; the machine-learning client module performs the API calls and renders the panel; and the machine-learning service module contains the Python code of the model.

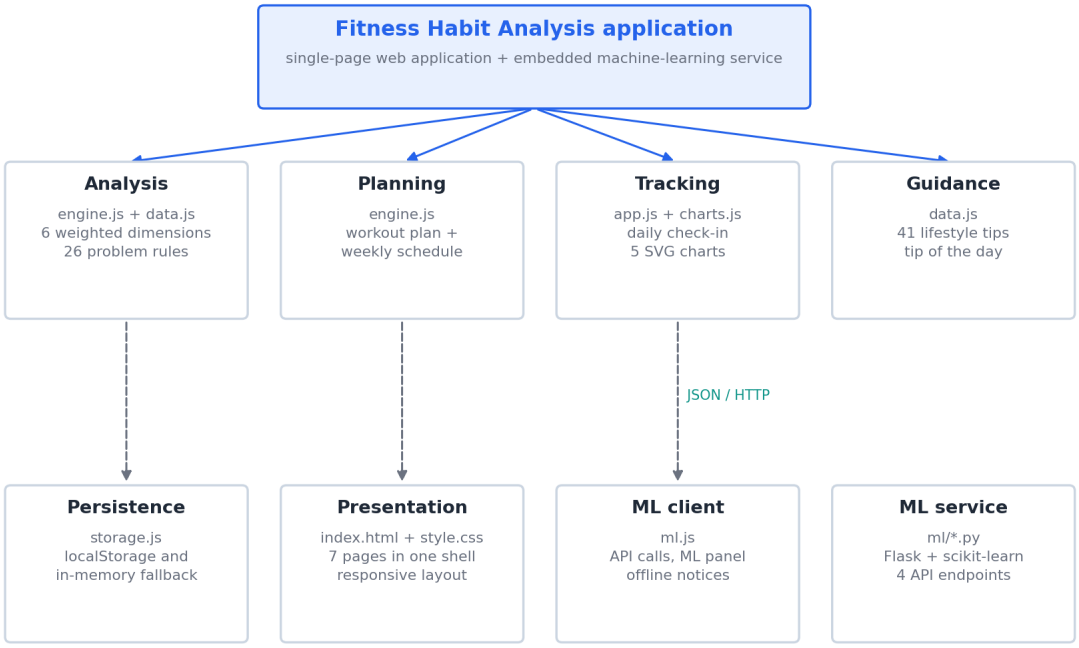



*Figure 2. Module structure of the application, showing the front-end modules and the machine-learning service.*

## 9.3. Data flow

A profile follows two paths that meet on the dashboard, as shown in Figure 3. In the first path the profile is validated, stored in the browser and passed to the rule engine, which produces the scores, the problems, the targets, the plan and the schedule for presentation. In the second path the same profile is posted to the prediction endpoint, converted into a feature frame and run through the fitted pipelines; the returned habit class, probabilities and ranked categories are rendered in the hybrid panel.

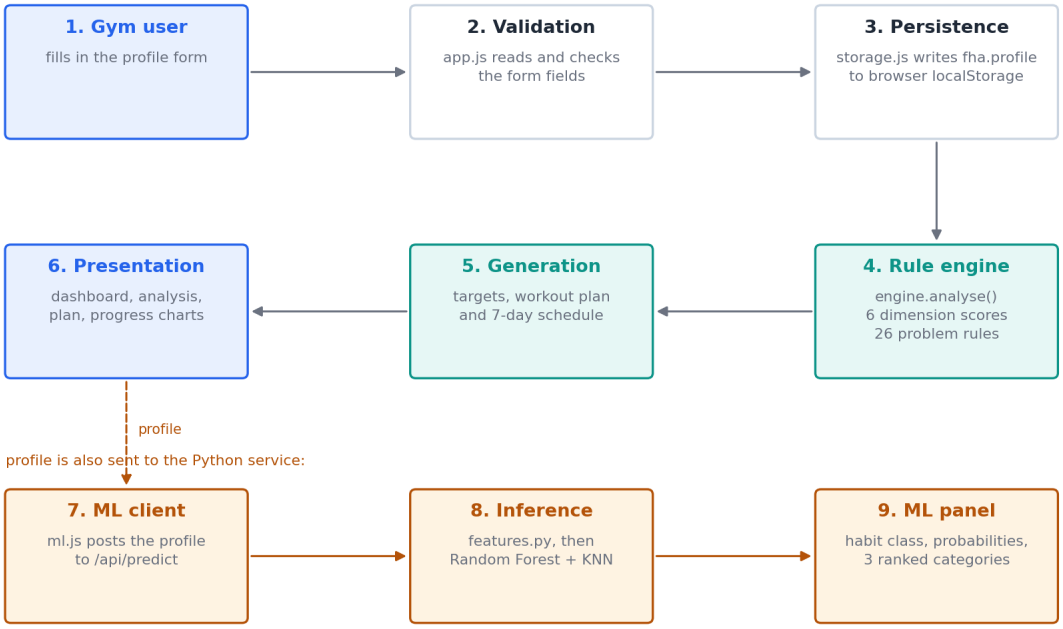



*Figure 3. Data flow of a user profile through validation, rule-based analysis and machine-learning inference.*

## 9.4. Deployment architecture

The whole system is deployed as one serverless function. The file app.py is the WSGI entrypoint that Vercel looks for; it inserts the project root into the module path, sets the read-only flag, and builds the Flask application without loading the model at import time. The routing configuration sends every request to that function, so the dashboard, the static assets and the API endpoints are served from the same process. Because the file system of a serverless deployment is read-only, the service loads the bundled model artefact, refuses to retrain through the API with a 403 response, and falls back to fitting the models in memory from the CSV when the artefact is missing or cannot be restored.

# CHAPTER 10
# DATASET DESCRIPTION

## 10.1. Source of the data

The training cohort is the file ml/data/fitness_habits.csv, which contains 1,733 labelled rows and 33 columns. It is built by the script ml/build_dataset.py in three steps. The first step downloads a public dataset of gym members, the Gym Members Exercise Dataset by Vala Khorasani from Kaggle, which contains 973 real workout sessions with age, gender, weight, height, body-mass index, body fat percentage, heart rates, session duration, calories burned, water intake, weekly training frequency, workout type and experience level. The raw file is cached in ml/data/raw and the download can be repeated at any time.

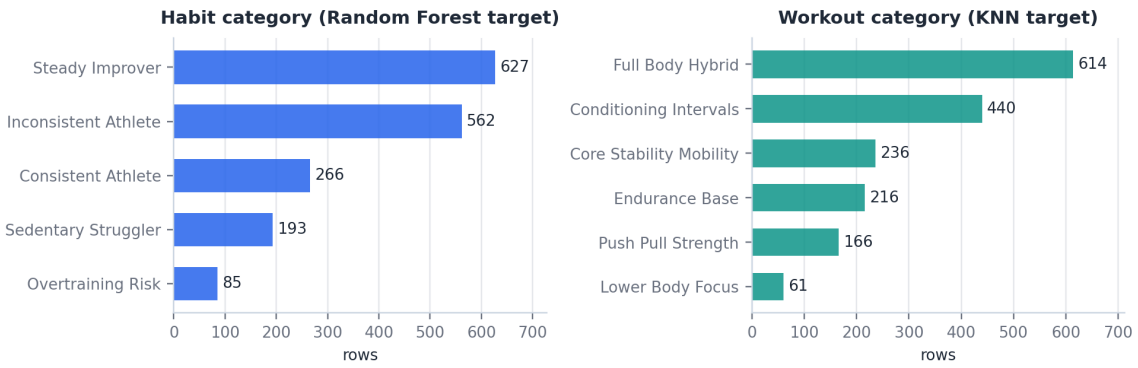
The second step simulates the lifestyle attributes that the public source does not contain - the training goal, motivation, diet quality, protein meals, junk food, sleep, daily steps, planned days, skipped sessions and the two tracking check-boxes - with a seeded random generator. The simulation is conditioned on the real row, so a trainee with a high body fat percentage receives a fat-loss goal more often than a lean trainee, and the planned session length is snapped to the same options that the profile form offers. The third step appends 760 fully simulated rows so that the rarer habit classes and the home and outdoor equipment profiles are represented as well. The provenance of every row is recorded in the row_source column as either real_public_dataset or simulated_cohort.

## 10.2. Class distribution

Both target labels were produced by transparent rules and then given a small amount of label noise, six percent for the habit label and four percent for the recommendation label, so that the learning task is not perfectly separable. The resulting class distributions are shown in Table 10.1 and Figure 4.

*Table 10.1 Class distribution of the training cohort*

| Habit category | Rows | Workout category | Rows |
|---|---|---|---|
| Steady improver | 627 | Full body hybrid | 614 |
| Inconsistent athlete | 562 | Conditioning intervals | 440 |
| Consistent athlete | 266 | Core and mobility | 236 |
| Sedentary struggler | 193 | Endurance base | 216 |
| Overtraining risk | 85 | Push and pull strength | 166 |
| | | Lower body focus | 61 |




Source: ml/artifacts/metrics.json — dataset distribution (1,733 rows).

*Figure 4. Distribution of the habit labels and of the workout labels in the 1,733-row training cohort.*

## 10.3. Data dictionary

Table 10.2 lists the columns of the cohort with their origin and their meaning. The last five columns are administrative: source_workout_type keeps the original workout type of the public data, row_source records the provenance of the row, and the remaining two columns hold the target labels.

*Table 10.2 Data dictionary of the labelled cohort*

| Column | Type | Meaning and origin |
|---|---|---|
| age, sex, height_cm, weight_kg, bmi | numeric, categorical | Body metrics taken from the public dataset |
| body_fat_pct, resting_bpm, avg_bpm, max_bpm | numeric | Physiological measurements from the public dataset |
| session_duration_min, calories_burned | numeric | Session measurements from the public dataset |
| water_glasses, workouts_per_week, experience_level | categorical and numeric | Weekly habits and experience, taken from the public dataset |
| source_workout_type | categorical | The workout type recorded in the public dataset |
| planned_days_per_week, skipped_per_week, session_minutes | numeric | Planning behaviour, simulated and conditioned on the real row |
| fitness_goal, equipment, activity_level, preferred_focus | categorical | Goal and preference attributes, simulated |
| diet_quality, protein_meals, junk_food_per_week, sleep_hours, daily_steps, resting_hr | ordinal, numeric | Lifestyle habits, simulated |
| screen_before_bed, tracks_progress | binary | The two check-boxes of the profile form, simulated |
| row_source | categorical | real_public_dataset or simulated_cohort |
| habit_category | categorical (target 1) | Label produced by the habit rules of ml/labels.py |
| recommended_category | categorical (target 2) | Label produced by the recommendation rules of ml/labels.py |

## 10.4. Missing values

A share of the cells was deliberately blanked in the generated CSV so that the imputation path of the preprocessing is always exercised: 863 cells in total. Table 10.3 lists the columns that contain missing values. The train-test split and the model selection therefore always run through a pipeline that fills the gaps rather than through a dataset that happens to be complete.

*Table 10.3 Missing cells per column*

| Column | Missing cells | Column | Missing cells |
|---|---|---|---|
| session_minutes | 96 | sleep_hours | 88 |
| skipped_per_week | 90 | daily_steps | 88 |
| diet_quality | 77 | protein_meals | 84 |
| junk_food_per_week | 80 | water_glasses | 82 |
| motivation | 67 | preferred_focus | 41 |
| equipment | 40 | activity_level | 30 |

## 10.5. Label generation rules

The habit label is produced by five ordered rules; the first rule that matches wins, as listed in Table 10.4. The adherence used by these rules is the ratio of completed sessions to planned sessions, capped at one.

*Table 10.4 Rules for the habit category*

| Order | Habit category | Condition |
|---|---|---|
| 1 | Sedentary struggler | One completed session or fewer per week |
| 2 | Overtraining risk | Five or more planned days, six or fewer hours of sleep and motivation of six or less |
| 3 | Consistent athlete | Adherence of at least 85 percent, at least seven hours of sleep and a diet quality of at least s |
| 4 | Steady improver | Adherence of at least 60 percent, at least six hours of sleep and a diet quality of at least five |
| 5 | Inconsistent athlete | Every other profile |

The recommendation label is produced by nine ordered rules over the goal, the experience level, the weekly frequency, the session length and the stated training preference; they are listed in Table 10.5.

*Table 10.5 Rules for the workout category*

| Goal | Condition | Workout category |
|---|---|---|
| Endurance | Three or fewer sessions and 45 minutes or longer | Endurance base |
| Endurance | Otherwise | Conditioning intervals |
| Muscle | Intermediate or advanced with four or more sessions | Push and pull strength |
| Muscle | Advanced with three or fewer sessions | Lower body focus |
| Muscle | Otherwise | Full body hybrid |
| Loss | Four or more sessions or a high-intensity preference | Conditioning intervals |
| Loss | Otherwise | Full body hybrid |
| Health | Mobility preference or 45 minutes or shorter | Core and mobility |
| Health | Otherwise | Full body hybrid |

# CHAPTER 11
# DATA PREPROCESSING

## 11.1. Validation before training

Before any preprocessing runs, the dataset module opens the CSV and validates it: the file must exist, it must be readable, it must contain every required column, it must have at least 80 labelled rows, every target must contain at least two classes, and no class may contain fewer than 15 rows. If any of these conditions fails, the service returns an insufficient-data message that names the failing condition instead of producing a prediction. The thresholds can be changed through environment variables for experiments.

## 11.2. Imputation

Missing values are filled by imputers inside the pipeline. Numeric columns are filled with the median of the column and the two binary check-box columns as well as the categorical columns are filled with the most frequent value. Because the imputers are part of the fitted pipeline, the statistics used at training time are also the statistics used at inference time.

## 11.3. Encoding of categorical features

The six categorical columns of the classifier - sex, fitness goal, experience level, equipment, activity level and preferred training focus - are one-hot encoded with an encoder that ignores unknown categories, so a value that never appeared during training cannot break the prediction. The recommender encodes its five categorical columns in the same way.

## 11.4. Scaling

The Random Forest does not need scaled inputs, so its numeric branch is left untouched apart from the imputation. The recommender, however, measures the euclidean distance between profiles; without scaling a daily step count of several thousand would dominate the distance and every neighbour would simply be the person who walks the most. The three numeric columns of the recommender - workouts per week, planned days per week and session minutes - are therefore standardised to zero mean and unit variance.

## 11.5. Train and test split

Each model is evaluated on a stratified split with a test fraction of 0.2 and the random seed 42, which leaves 1,386 training rows and 347 test rows for every model, the class proportions of the full cohort being preserved in both parts. A five-fold stratified cross-validation over the full cohort

provides the stability estimate that is reported beside the test accuracy.

# CHAPTER 12

# ACTUAL MACHINE LEARNING ALGORITHM AND MODEL WORKFLOW

## 12.1. Choice of algorithms

Two classic supervised algorithms were chosen. The habit classification is a five-class problem over a mixed feature vector of numeric, binary and categorical columns with a strongly imbalanced target, which is exactly the setting a Random Forest handles well: it performs implicit feature selection, tolerates irrelevant columns and can be rebalanced with class weights. The workout recommendation is a similarity problem - the question is which previously seen trainee is most similar to this one - and the k nearest neighbours algorithm answers that question directly while remaining simple enough to explain during a project defence.

## 12.2. Random Forest classifier

The classifier is a scikit-learn RandomForestClassifier with 300 trees, a maximum depth of 10, a minimum of two samples per leaf, the square-root feature sampling, balanced class weights and the random seed 42; the estimators are trained on all available cores. It consumes the 23 features of the habit profile - fifteen numeric columns, the two check-boxes and the six categorical columns - and predicts one of the five habit categories together with the probability of every class. The parameters are listed in Table 12.1.

*Table 12.1 Hyperparameters of the fitted models*

| Model | Parameter | Value |
|---|---|---|
| Random Forest | n_estimators | 300 |
| Random Forest | max_depth | 10 |
| Random Forest | min_samples_leaf | 2 |
| Random Forest | max_features | sqrt |
| Random Forest | class_weight | balanced |
| Random Forest | random_state | 42 |
| K nearest neighbours | n_neighbors | 9 |
| K nearest neighbours | weights | distance |
| K nearest neighbours | p | 2 (euclidean) |
| Both models | test_size | 0.2 (stratified, seed 42) |
| Both models | cross-validation | 5-fold stratified |

## 12.3. K nearest neighbours recommender

The recommender is a scikit-learn KNeighborsClassifier with nine neighbours and distance weighting, which means that a closer training profile contributes more to the vote than a distant one. It receives eight features - the three numeric columns that are standardised and the five categorical columns that are one-hot encoded. At inference time the service reads the predicted probabilities as the match scores of the workout categories, keeps the top three, and additionally returns the labels and distances of the nine nearest training profiles so that the interface can show where the suggestion came from.

## 12.4. Model workflow

The complete workflow, from the raw public file to the panel in the browser, is shown in Figure 5. The dataset builder downloads and normalises the public cohort, simulates the lifestyle columns and the extra rows, applies both label rules with controlled noise and blanks a share of the cells. The dataset module validates the CSV, the split separates training and test rows, the column transformers impute, encode and scale, both estimators are fitted, the evaluation computes the metrics, and the fitted pipelines and the report are written as model.joblib and metrics.json. The Flask API then loads the artefact and answers the prediction request of the dashboard.

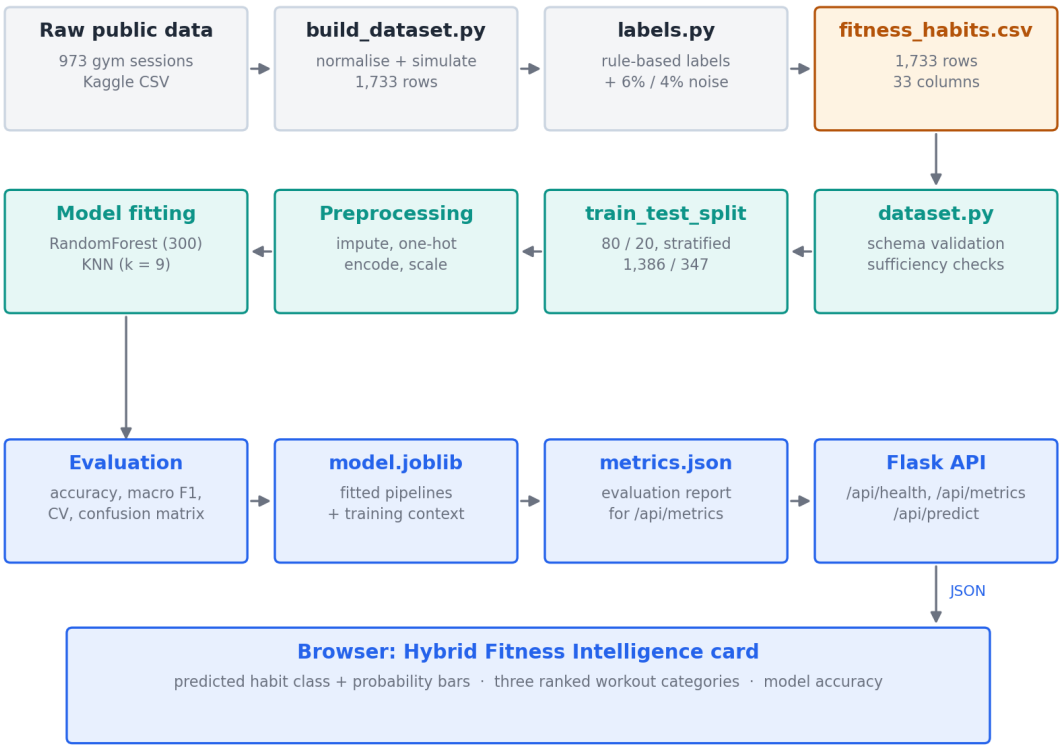



*Figure 5. End-to-end workflow of the machine-learning layer, from the public source to the panel rendered in the browser.*

## 12.5. Inference for a user profile

A prediction request carries the profile exactly as the form stores it. The feature module maps the form names to the column names of the cohort, derives the body-mass index from height and weight, computes a training preference when the user did not choose one, and reports every field that had to be defaulted so that the interface can stay transparent about it. The habit payload is the class with the highest forest probability together with the sorted probability list of all five classes; the recommendation payload is the sorted probability list of the six workout categories truncated to the top three, each with its focus description and a short data-driven reason sentence.

# CHAPTER 13
# IMPLEMENTATION DETAILS

## 13.1. Implementation of the front end

The front end is plain HTML, CSS and JavaScript. There is no framework, no bundler and no external library: the project runs by opening the file index.html or by serving the folder with any static server. Eight files make up the interface, listed in Table 13.1.

*Table 13.1 Files of the front end*

| File | Role |
|---|---|
| index.html | Single-page shell holding the seven pages and the navigation |
| assets/css/style.css | Design system with tokens, components and responsive rules |
| assets/js/data.js | Content library: 58 exercises, 24 session templates, prescriptions, 41 tips, checklist items and the |
| assets/js/storage.js | Wrapper around the browser storage with an in-memory fallback |
| assets/js/engine.js | Rule engine: the six dimension scores, the 26 problem rules, the targets, the plan and the schedu |
| assets/js/charts.js | Dependency-free SVG charts for the progress page and the panels |
| assets/js/ml.js | Client of the prediction API and renderer of the hybrid panel |
| assets/js/app.js | Controller: navigation, forms, events, rendering and sample data |

The analysis module computes the six scores with fixed weights - consistency 25 percent, nutrition 20 percent, and hydration, sleep, activity and motivation 15, 15, 15 and 10 percent - and combines them into the overall score whose bands are Excellent from 75, Good from 60, Needs attention from 40 and Poor below 40. The plan generator chooses the training split for the goal, maps the session types onto the training days, resolves an equipment-appropriate and beginner-safe movement for every exercise, sizes the session to the time budget, and applies the sets, repetitions and rest prescription of the goal and the experience level.

## 13.2. Implementation of the machine-learning service

The service is a Python package of eleven modules, listed in Table 13.2, that can be executed from the command line or imported by the WSGI entrypoint.

*Table 13.2 Modules of the machine-learning package*

| Module | Responsibility |
|---|---|
| ml/config.py | Paths, feature lists, label vocabulary, thresholds and environment overrides |
| ml/labels.py | The transparent labelling rules for both targets and the presentation helpers |
| ml/build_dataset.py | Downloads the public CSV, simulates the lifestyle columns and writes the labelled cohort |
| ml/dataset.py | Loads the CSV, validates the schema and reports insufficient training data |
| ml/features.py | Maps a browser profile to a one-row feature frame and reports defaulted fields |
| ml/preprocess.py | The column transformers with imputation, encoding and scaling |

| | |
|---|---|
| ml/train.py | Trains both models, computes the metrics and writes the artefacts |
| ml/predict.py | Runs inference for a profile and builds the API payload |
| ml/serve.py | Flask application with the API blueprint, the static route and the model lifecycle |
| app.py | WSGI entrypoint used by Vercel and by any WSGI server |
| ml/tests/test_model.py | The automated test suite |

## 13.3. The application programming interface

The service exposes four endpoints on the same origin as the page, listed in Table 13.3, so the browser does not need any cross-origin configuration. A health check runs before every request and fits the model on the first call, which keeps a cold start short.

*Table 13.3 Endpoints of the JSON API*

| Method | Endpoint | Response |
|---|---|---|
| GET | /api/health | Service status, dataset report and model summary |
| GET | /api/metrics | The complete evaluation report stored in metrics.json |
| POST | /api/predict | Habit class with probabilities, top three recommendations and neighbour labels |
| POST | /api/train | Retrains both models; refused with 403 on a read-only deployment |

## 13.4. Protection of the deployed files

The static route serves only the file types of the web interface and only from the shell and the assets folder. Python source files, the training CSV, the fitted model, the dependency files and the routing configuration therefore return a 404 response on the deployed application, which was verified against the live deployment. The read-only flag additionally blocks retraining so that an anonymous request cannot spend the CPU of the instance.

## 13.5. Screenshots of the running application

The following screenshots were captured from the running application with the clearly labelled sample data loaded, so that the dashboard, the analysis, the plan, the progress charts and the hybrid panel are shown with content.

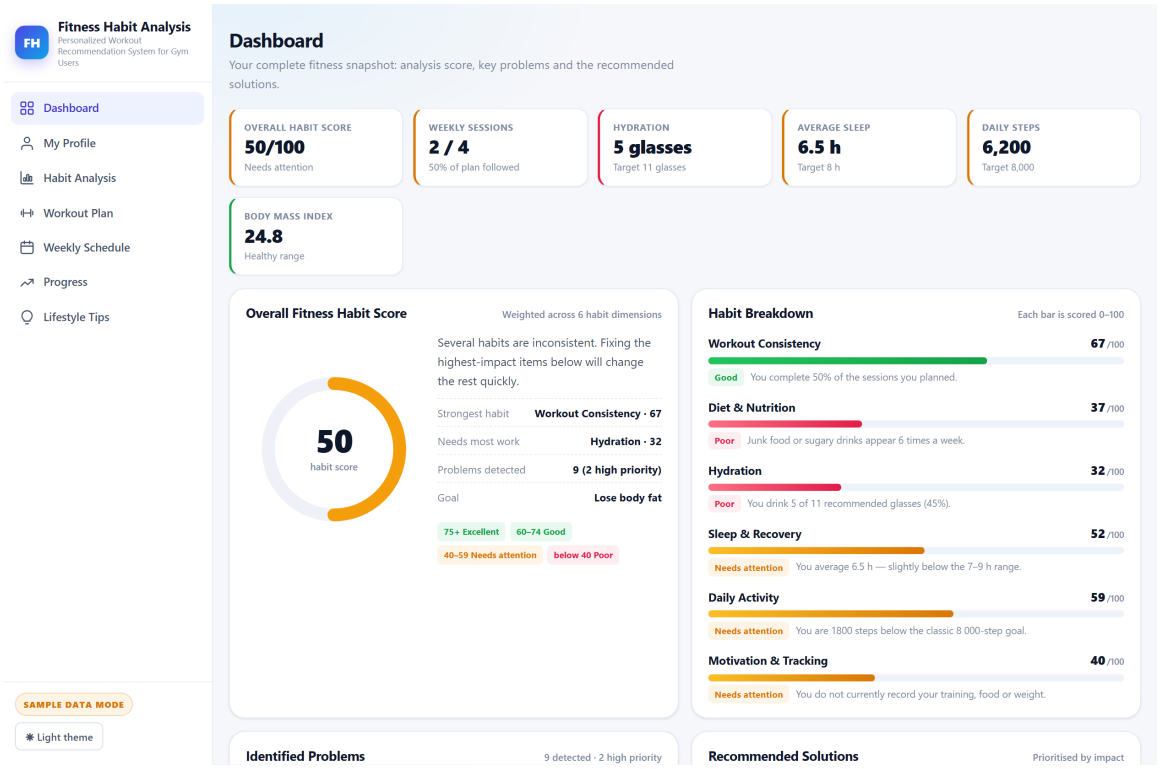



*Figure 6. Dashboard with the habit score gauge, the dimension breakdown, the detected problems and the ranked solutions.*

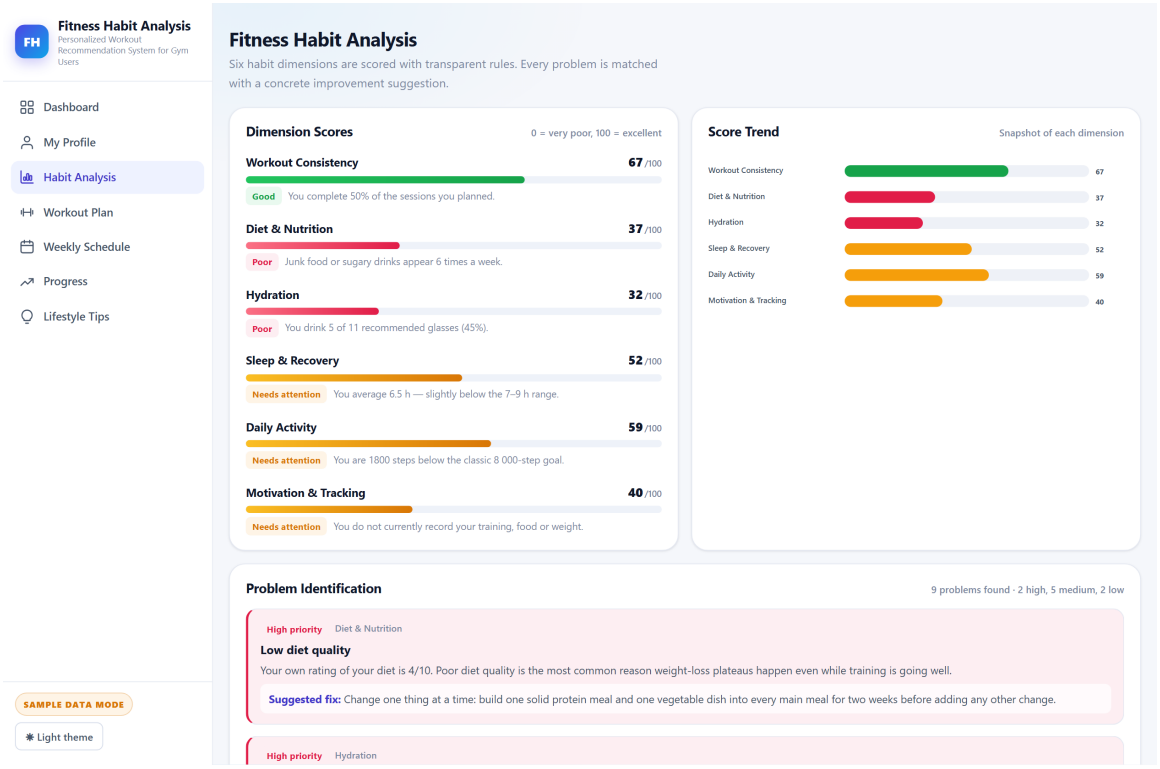



*Figure 7. Analysis page showing the six habit dimensions with their status bands.*

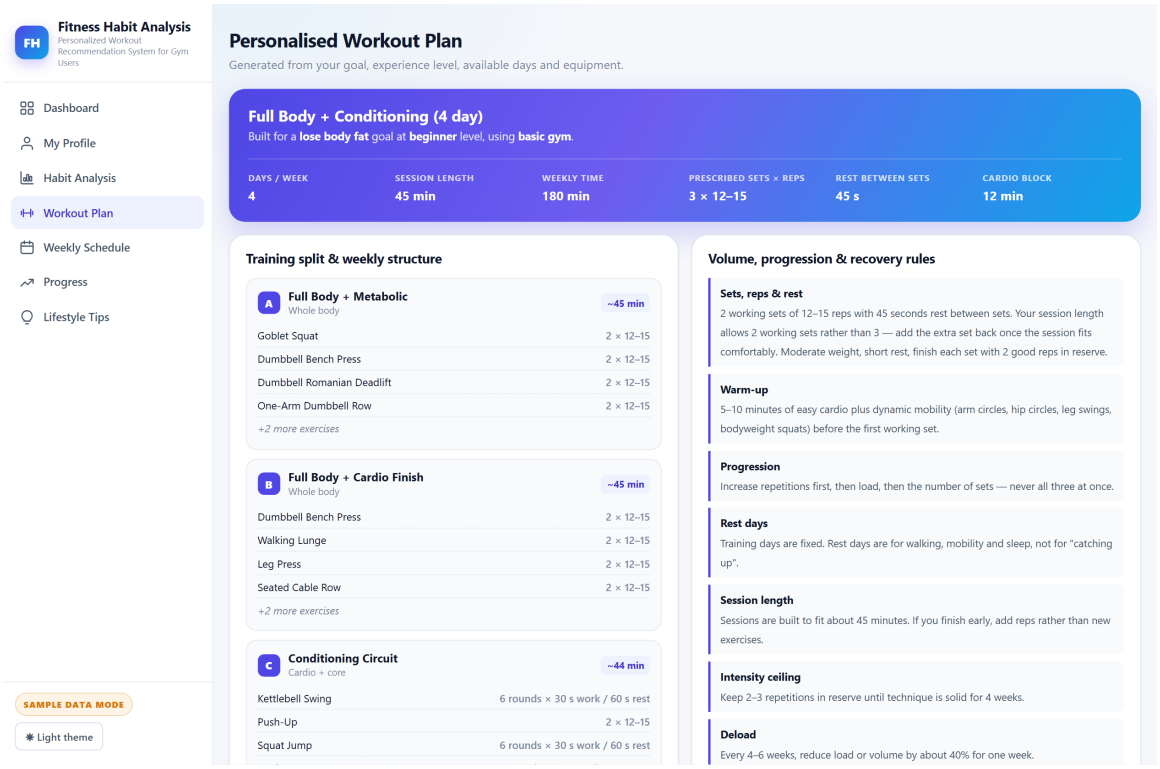



*Figure 8. Personalised workout plan generated for the profile, including exercises, sets and rest.*

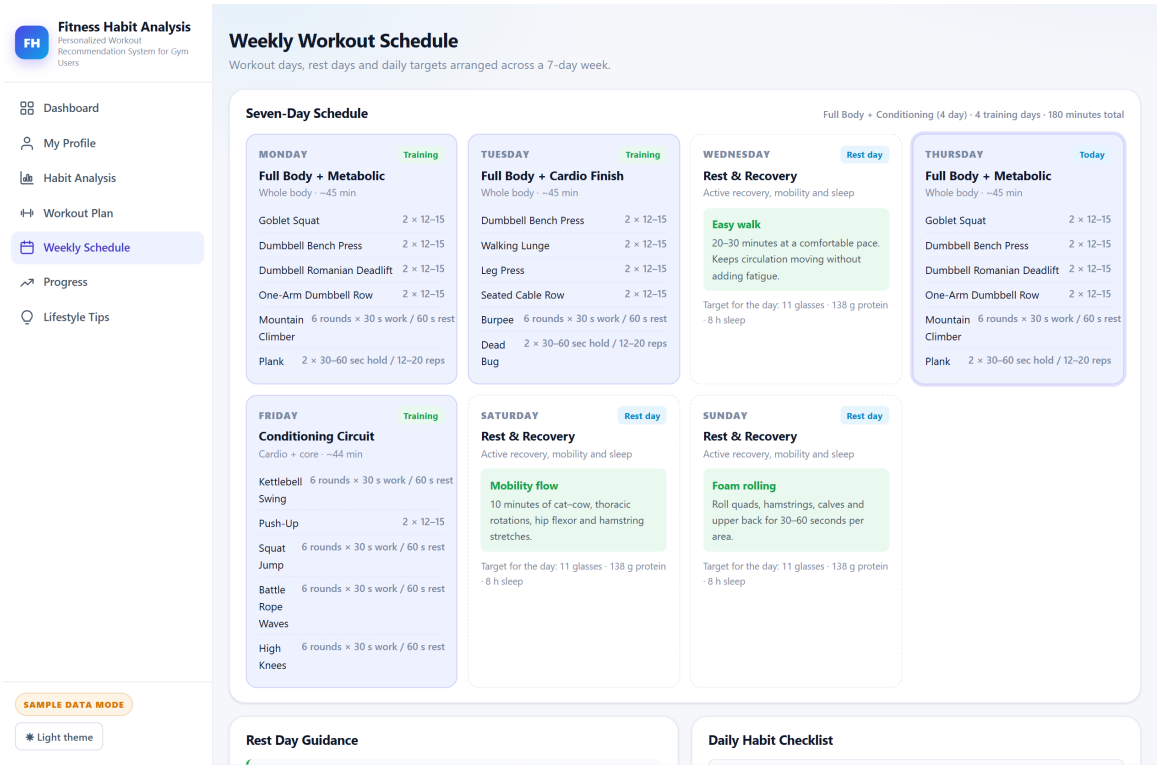



*Figure 9. Seven-day schedule with training days, session types and recovery activities for the rest days.*

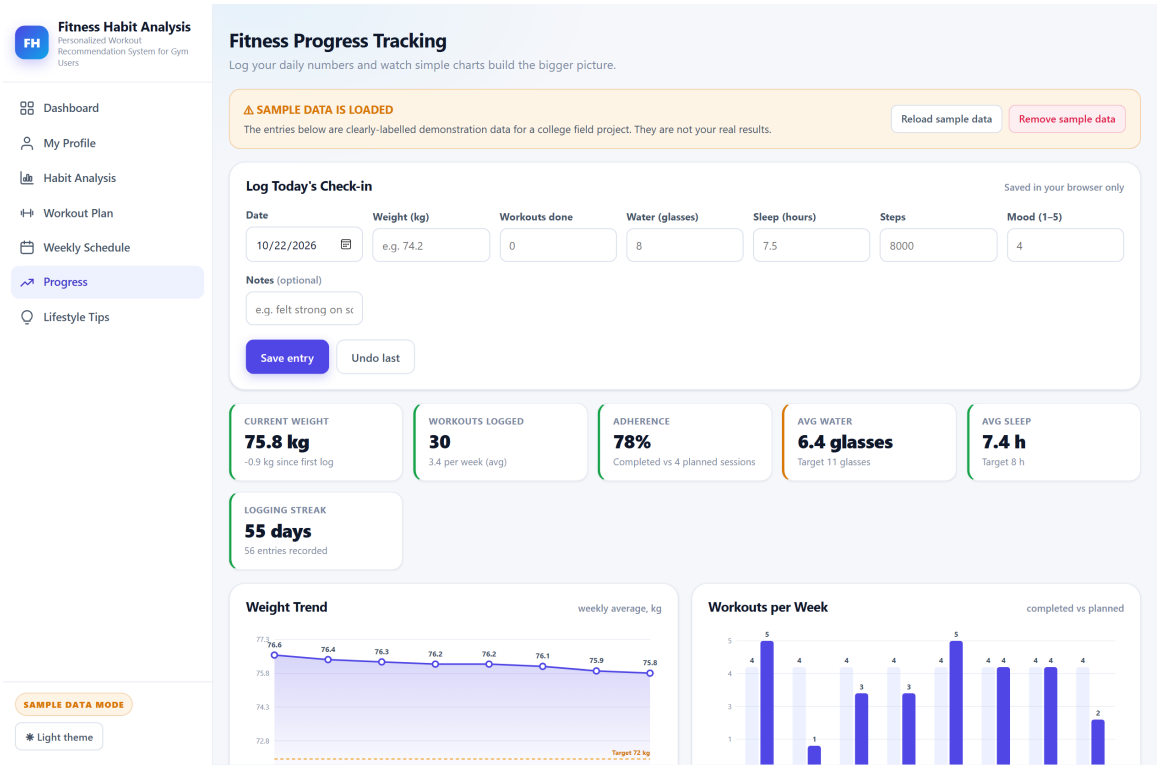



*Figure 10. Progress page with the daily check-in and the SVG charts of weight, workouts, hydration and sleep.*

# CHAPTER 14

# RESULTS AND ANALYSIS

## 14.1. Evaluation methodology

Both models were trained and evaluated by the script ml/train.py on the cohort of 1,733 rows. Every model received its own stratified 80/20 split, which produced 1,386 training rows and 347 test rows, and a five-fold stratified cross-validation over the full cohort was run for the stability estimate. Accuracy, macro and weighted precision, recall and F1, the confusion matrix, the per-class report and a majority-class baseline computed from the training split were written to ml/artifacts/metrics.json, which the API serves live at GET /api/metrics.

## 14.2. Results of the Random Forest classifier

Table 14.1 lists the headline metrics of the habit classifier. The test accuracy of 0.8876 exceeds the majority-class baseline of 0.3631 by more than fifty-two percentage points, and the cross-validation accuracy of 0.8656 with a standard deviation of 0.0184 shows that the result is stable across folds rather than a lucky split.

*Table 14.1 Evaluation metrics of the Random Forest classifier*

| Metric | Value |
|---|---|
| Test accuracy | 0.8876 |
| Macro precision | 0.8523 |
| Macro recall | 0.8225 |
| Macro F1 | 0.8277 |
| Weighted F1 | 0.8847 |
| Five-fold cross-validation accuracy | 0.8656 (standard deviation 0.0184) |
| Majority-class baseline | 0.3631 |
| Training rows / test rows | 1,386 / 347 |

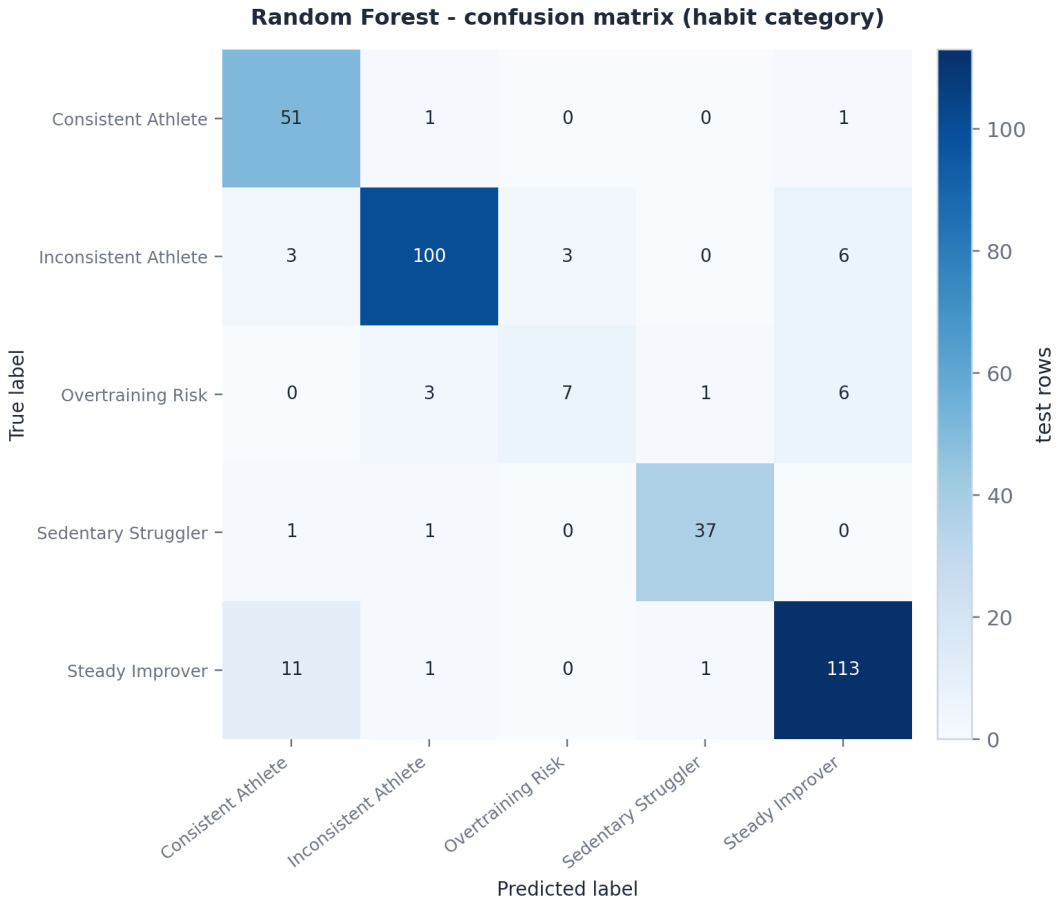



*Figure 11. Confusion matrix of the Random Forest classifier on the 347 test rows.*

The per-class behaviour of the classifier is shown in Table 14.2. The classes with a large number of rows are predicted almost perfectly: consistent athlete 0.96, sedentary struggler 0.95, steady improver 0.90 and inconsistent athlete 0.89 recall. The rare class is the weak point: overtraining risk has only 85 rows in the whole cohort and 17 in the test split, and its recall of 0.41 means that most of its test rows are confused with consistent athlete and steady improver - the classes it resembles when the adherence happens to be high.

*Table 14.2 Per-class report of the habit classifier*

| Habit class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| Consistent athlete | 0.77 | 0.96 | 0.86 | 53 |
| Inconsistent athlete | 0.94 | 0.89 | 0.92 | 112 |
| Overtraining risk | 0.70 | 0.41 | 0.52 | 17 |
| Sedentary struggler | 0.95 | 0.95 | 0.95 | 39 |
| Steady improver | 0.90 | 0.90 | 0.90 | 126 |
| Macro average | 0.85 | 0.82 | 0.83 | 347 |
| Weighted average | 0.89 | 0.89 | 0.88 | 347 |

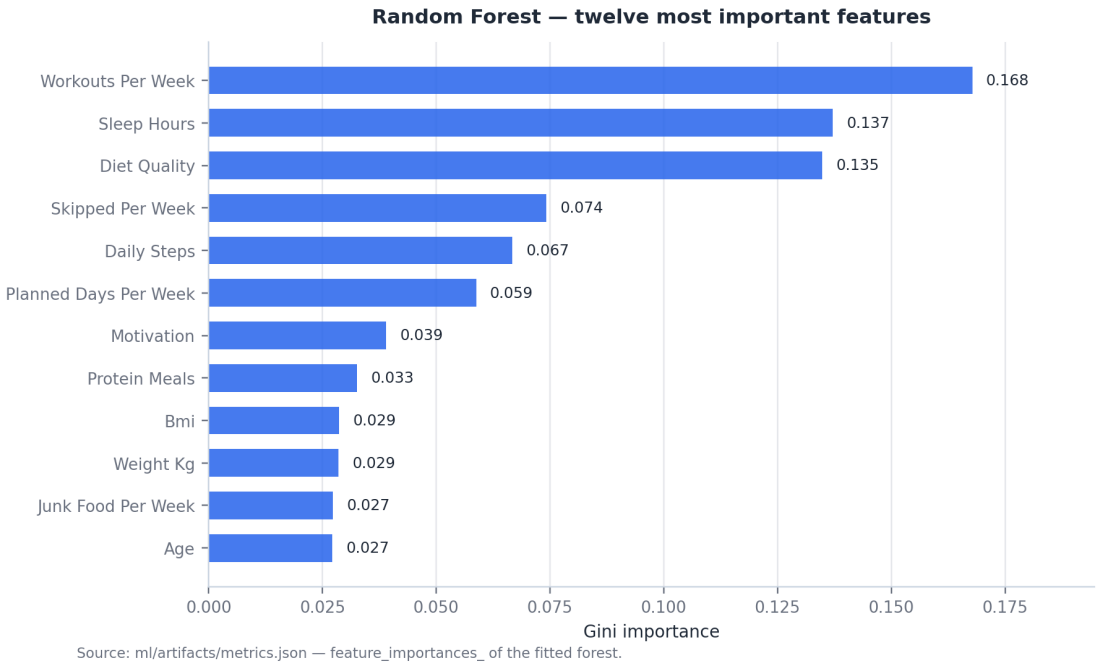



*Figure 12. The twelve most important features of the fitted forest, taken from the feature importances reported in metrics.json.*

The importances in Figure 12 confirm that the model learned the intended surface: training frequency is the strongest signal at 0.168, followed by sleep at 0.137 and diet quality at 0.135, then skipped sessions and daily steps. These are exactly the variables the labelling rules use, but the forest combines them with the remaining features and can therefore still disagree with an individual rule, which is what makes it a genuine second opinion instead of a lookup table.

## 14.3. Results of the K nearest neighbours recommender

Table 14.3 lists the headline metrics of the workout recommender. Its accuracy of 0.8588 and macro F1 of 0.7966 both exceed the majority-class baseline of 0.3545 by roughly fifty points, and the cross-validation accuracy of 0.8390 with a standard deviation of 0.0252 shows the expected slightly higher variance of a neighbour-based model.

*Table 14.3 Evaluation metrics of the K nearest neighbours recommender*

| Metric | Value |
|---|---|
| Test accuracy | 0.8588 |
| Macro precision | 0.8756 |
| Macro recall | 0.7750 |
| Macro F1 | 0.7966 |
| Weighted F1 | 0.8544 |
| Five-fold cross-validation accuracy | 0.8390 (standard deviation 0.0252) |
| Majority-class baseline | 0.3545 |
| Top-two hit rate | 0.9481 |
| Top-three hit rate | 0.9683 |

The top-two and top-three hit rates are the numbers that matter for the interface, because the dashboard shows three suggestions: on 96.8 percent of the test rows the correct workout category is among the three categories displayed. Figure 13 shows the confusion matrix, Figure 14 the per-class recall of both models, and those recall values are hybrid, 0.88 for push and pull strength, 0.86 for conditioning intervals, 0.83 for core and mobility, 0.81 for endurance base and 0.33 for lower body focus. Lower body focus is again the rare class with 61 rows in the cohort and only 12 in the test split; it is the category that suffers most from the label noise and from its similarity to the strength-oriented categories.

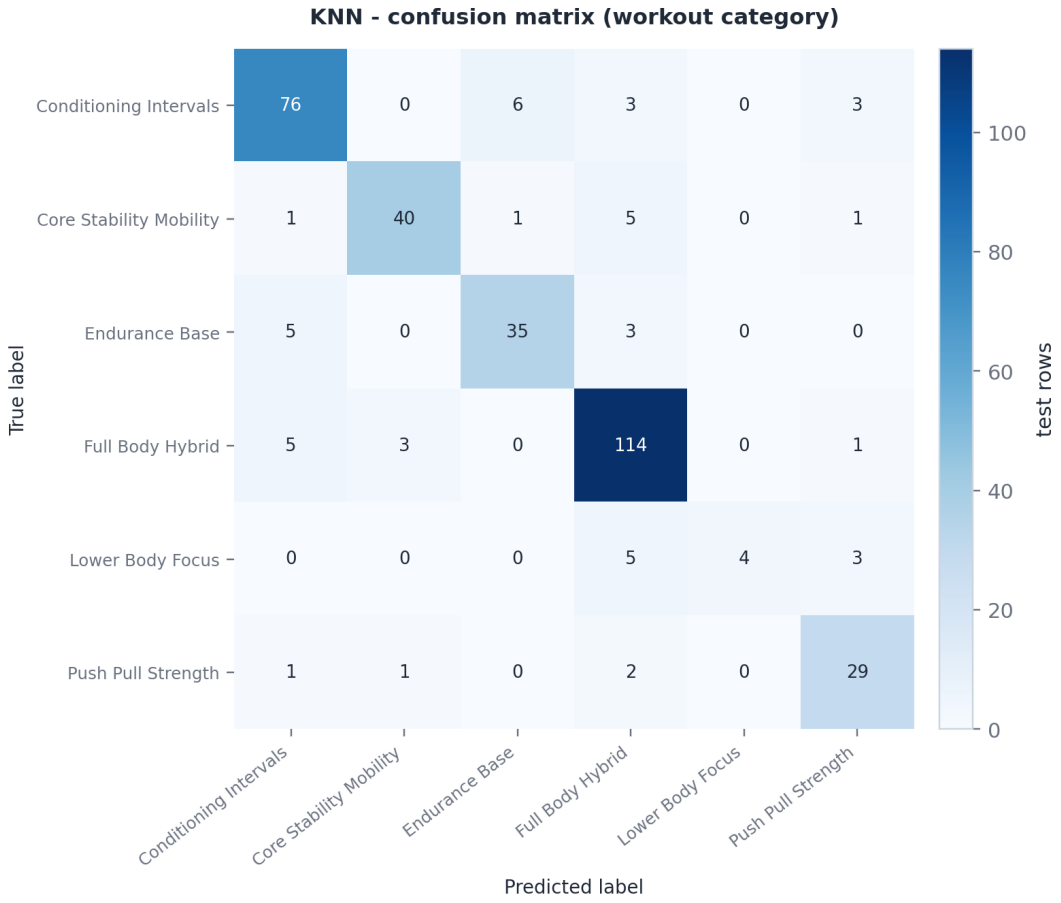
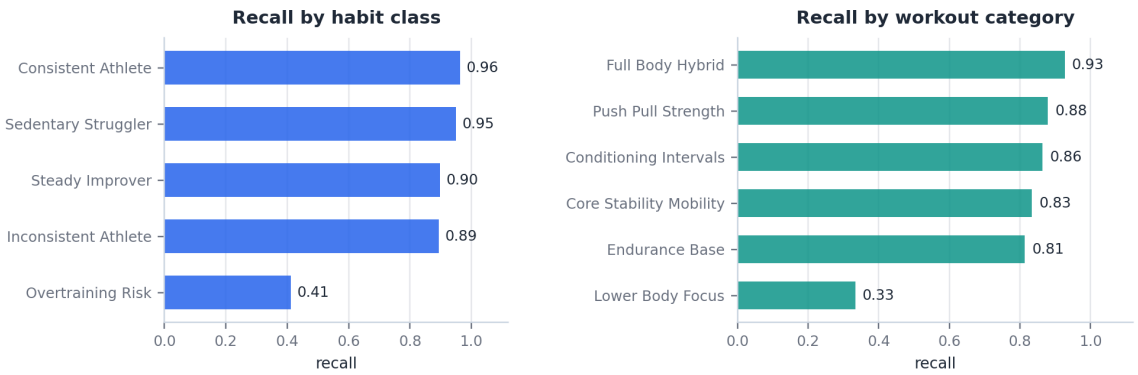



*Figure 13. Confusion matrix of the recommender on the 347 test rows.*

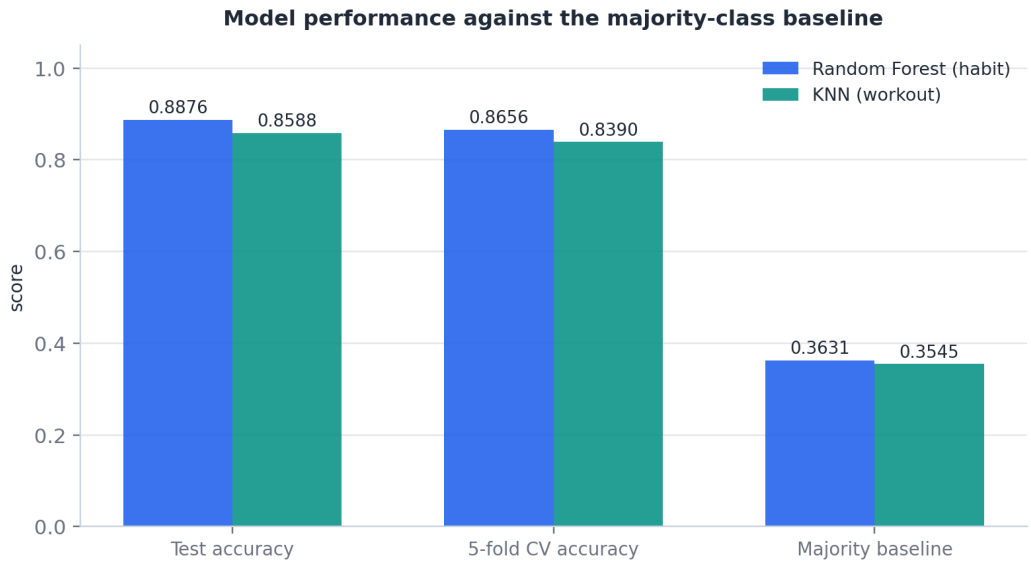



Source: ml/artifacts/metrics.json — per-class recall on the 347-row test split.

*Figure 14. Per-class recall of both models on the held-out test split.*

## 14.4. Comparison with the baseline

Figure 15 compares each model with its majority-class baseline. A model that always predicted the most frequent class would score 0.3631 and 0.3545 respectively, so both models roughly double that figure. The gap between the cross-validation accuracy and the test accuracy is smaller than one standard deviation for both models, which indicates that the single split used for the tables is representative.




Source: ml/artifacts/metrics.json (test set n = 347; 5-fold stratified CV).

*Figure 15. Test accuracy, cross-validation accuracy and majority-class baseline of both models.*

## 14.5. Result rendered in the application

Figure 16 shows the hybrid panel as the user sees it: the predicted habit category with its confidence, a probability bar for every habit class, the three ranked workout categories with their match scores, a short reason for every suggestion, the labels of the nearest training profiles, the model names, the number of training rows and the test accuracy of both models. The values are taken from the response of the API; the panel contains no local calculation of its own.

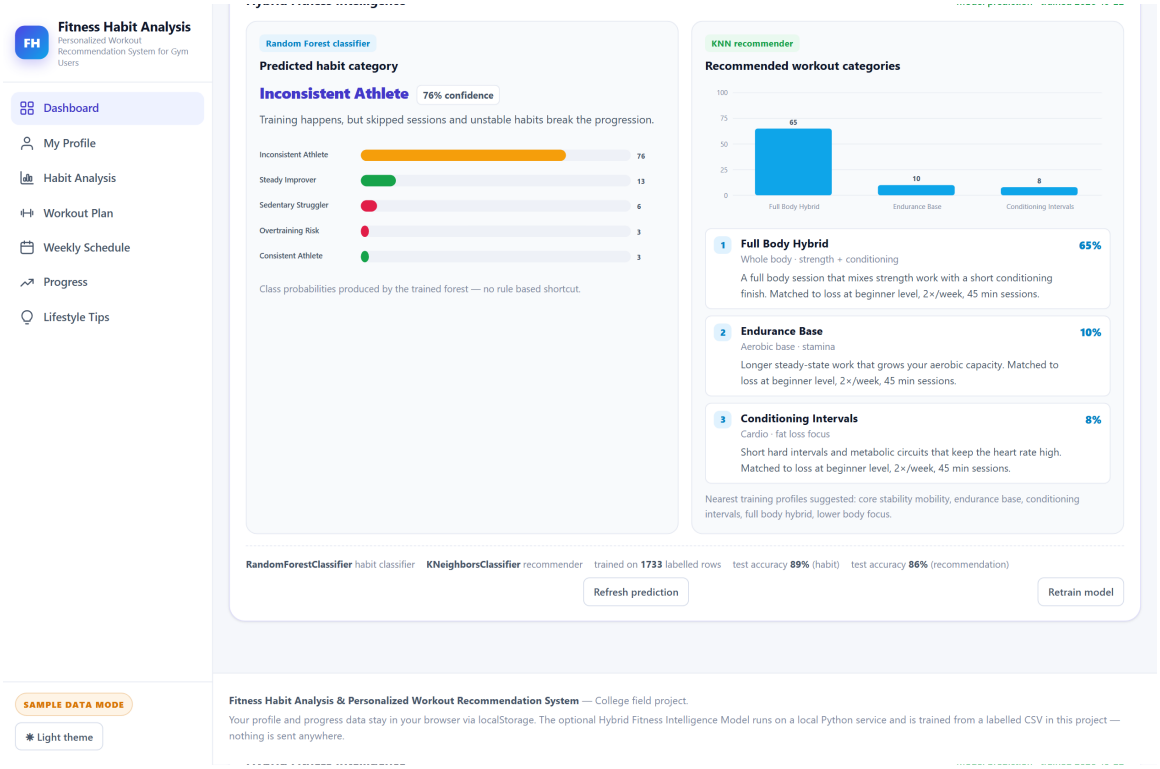



*Figure 16. The Hybrid Fitness Intelligence panel on the dashboard, showing the classification of the profile, the class probabilities and the ranked workout categories.*

## 14.6. Summary of the results

*Table 14.4 Summary of the results of both models*

| Model | Task | Test accuracy | Macro F1 | Cross-validation | Baseline |
|---|---|---|---|---|---|
| Random Forest | Habit category (5 classes) | 0.8876 | 0.8277 | 0.8656 | 0.3631 |
| K nearest neighbours | Workout category (6 classes) | 0.8588 | 0.7966 | 0.8390 | 0.3545 |

# CHAPTER 15

# TESTING

## 15.1. Strategy

The testing of the project has three layers: automated tests for the machine-learning package, a systematic sweep over the rule engine, and an integration test of the interface in a simulated browser. The commands that reproduce the reported results are shown below.

*Table 15.1 Test suites of the project*

| Suite | Command | Result |
|---|---|---|
| Machine-learning tests | python -m pytest ml/tests -q | 20 passed |
| Rule-engine sweep | Run in the browser console or from the test harness | 960 profile combinations, all passed |
| Interface integration | jsdom integration run of the full user journey | All passed with 0 console errors |

## 15.2. Automated test suite of the machine-learning package

The suite ml/tests/test_model.py contains 20 tests. They were executed together and all of them passed. Table 15.2 lists the tests with the property each one protects; the complete output of the run is the line 20 passed.

*Table 15.2 Automated tests of the machine-learning package*

| Test | Property under test |
|---|---|
| test_training_dataset_is_usable | The cohort has the required columns, enough rows and enough classes |
| test_training_dataset_contains_missing_values | The deliberate blanks are present so imputation runs |
| test_insufficient_data_is_reported | A tiny or broken CSV produces a clear insufficient-data message |
| test_label_rules_are_deterministic | Both label rules return the expected class for known profiles |
| test_profile_maps_to_model_features | The browser profile becomes the exact feature frame of the model |
| test_profile_defaults_are_reported_when_fields_are_missing | Missing fields are defaulted and reported |
| test_preprocessor_imputes_missing_values | The imputer fills the blanks of the numeric columns |
| test_knn_preprocessor_scales_numeric_features | The recommender receives scaled numeric inputs |
| test_models_reach_useful_accuracy | Both models score well above their baselines |
| test_prediction_returns_class_and_recommendations | The API payload contains a class and three recommendations |
| test_prediction_is_not_hardcoded | Two different profiles produce two different predictions |
| test_api_health | The health endpoint reports the dataset and the model |
| test_api_predict | The predict endpoint answers with a valid payload |
| test_api_metrics | The metrics endpoint serves the evaluation report |
| test_api_reports_insufficient_data | The API reports insufficient data instead of guessing |
| test_static_frontend_is_served | The shell of the application is served |
| test_root_entrypoint_exposes_a_wsgi_app | app.py exposes the WSGI application that Vercel expects |

| | |
|---|---|
| test_readonly_deployment_trains_in_memory | A read-only file system still yields a fitted model |
| test_retraining_is_refused_on_a_readonly_deployment | Retraining is blocked with a 403 response |
| test_broken_artifacts_are_detected_and_retrained | An unusable artefact triggers a retrain |

### 15.3. Testing of the rule engine

The plan generator was exercised over 960 profile combinations - four goals, three experience levels, four equipment types, five training-day counts and four session lengths - and every produced plan was checked for a valid session, for movements that the chosen equipment actually supports and for a schedule with real rest days. All combinations passed.

### 15.4. Testing of the interface

The interface was tested as an integration flow in a simulated DOM: submitting the profile, opening the analysis, generating the plan and the schedule, logging a check-in, rendering the charts, deleting an entry with undo, loading and removing the sample data, navigating between the pages, clearing the data and reloading the browser to confirm that everything is restored from the storage. The run finished with no console errors.

### 15.5. Testing of the deployed application

The deployed application was verified over HTTP against the production URL: the page returns the status code 200, the health endpoint reports the status ok with 1,733 rows and the accuracies of the two models, a prediction request returns the habit class with its probabilities and the three recommendations, and the protected paths - the Python source, the training CSV, the model artefact, the dependency file and the routing configuration - return 404. Retraining was confirmed to be refused with a 403 response on the read-only deployment.

# CHAPTER 16
# LIMITATIONS

## 16.1. Limitations of the implementation

• The lifestyle columns of the cohort are simulated. The base rows are real, but no public dataset combines physiological measurements with goal, motivation, diet, sleep and tracking habits, so the labels describe the documented rule set rather than measured clinical outcomes.

• The habit classifier is a descriptive aid. It summarises the same profile the rule engine already analyses, from a different angle, and it is not a medical device.

• The rare classes are hard. Overtraining risk has 85 rows and lower body focus has 61, and their recall - 0.41 and 0.33 - limits how far the two models can be trusted for those profiles.

• The recommender suggests a category of training, not individual exercises; the concrete sessions still come from the rule-based plan generator.

• Everything runs on the machine of the user. There are no accounts, no synchronisation between devices and no analytics of what the recommendations achieve over time.

• The label rules encode the fitness knowledge of the project author. A different expert set would produce different labels and therefore different models.

• The evaluation is offline. No user study has yet measured whether following the recommendation actually improves adherence.

# CHAPTER 17

# FUTURE SCOPE

## 17.1. Planned extensions

• A habit heat-map calendar view that shows adherence over months at a glance.

• Export of the complete workout plan to a printable document or a PDF.

• Re-assessment of the goal and the plan after a fixed number of weeks, driven by the progress log.

• A larger exercise library with machine-specific and alternative movements.

• Optional comparison of adherence between users of the same club, with strict opt-in.

## 17.2. Extensions of the machine-learning layer

• Collecting a real labelled cohort, either by a survey of gym members or in cooperation with a club, to replace the simulated lifestyle columns with measured data.

• Class-weighted or cost-sensitive training for the rare classes, or a merge of the two smallest classes into one, to raise the recall of overtraining risk and lower body focus.

• Comparing the Random Forest with gradient boosting and with regularised linear models on the same split to confirm that the choice was justified.

• Calibrating the class probabilities so that the confidence shown in the panel can be read as a real probability.

• A sequence model over the daily check-ins, which would turn the classification from a snapshot into a trend over weeks.

• Persisting predictions per user to evaluate whether following the recommendation improves the habit scores.

# CHAPTER 18
# CONCLUSION

## 18.1. Conclusion

The project delivered a complete fitness habit analysis and personalised workout recommendation system for gym users. The application analyses the profile of a user across six weighted habit dimensions, detects problems with 26 ranked rules, derives personal nutrition and activity targets, generates a personalised workout plan and a seven-day schedule from a library of 58 exercises, tracks progress with five charts and offers 41 lifestyle tips - all of it in a browser with no framework, no build step and no server-side storage of user data.

On top of that foundation, the hybrid machine-learning layer classifies the habit profile of the trainee with a Random Forest that reaches a test accuracy of 0.8876 against a baseline of 0.3631, and recommends three workout categories with a nine-neighbour model that reaches 0.8588 accuracy and a top-three hit rate of 0.9683 against a baseline of 0.3545. Both models are wrapped in preprocessing pipelines that impute the 863 blank cells, encode the categorical columns and scale the numeric inputs of the recommender, and both are evaluated with a stratified split, five-fold cross-validation, per-class reports and confusion matrices.

The result is served by a small JSON API, displayed on the dashboard with its confidence and its evidence, and it degrades honestly: when the service is not reachable or the labelled data is too small, the interface says so instead of inventing an answer. The system is covered by 20 automated tests, a 960-combination sweep of the plan generator and a full interface integration run, and it is deployed as a read-only serverless application. The work demonstrates that a transparent rule engine and a supervised model can be combined in one product without sacrificing either explainability or measured performance.

# CHAPTER 19
# REFERENCES

## 19.1. References

[1] V. Khorasani, Gym Members Exercise Dataset, Kaggle. https://www.kaggle.com/datasets/valakhorasani/gym-members-exercise-dataset

[2] Scikit-learn developers, Scikit-learn: Machine Learning in Python, https://scikit-learn.org/stable/documentation.html

[3] L. Buitinck et al., API design for machine learning software: experiences from the scikit-learn project, arXiv:1309.0238, 2013.

[4] P. Pedregosa et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] The pandas development team, pandas - pandas documentation, https://pandas.pydata.org/docs/

[6] Flask contributors, Flask Documentation, https://flask.palletsprojects.com/

[7] joblib developers, joblib - running Python functions as pipeline jobs, https://joblib.readthedocs.io/

[8] NumPy developers, NumPy documentation, https://numpy.org/doc/stable/

[9] Vercel, Flask deployment on Vercel, https://vercel.com/docs/frameworks/backend/flask

[10] MDN Web Docs, Window: localStorage property, https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[11] M. Mifflin, S. St Jeor, L. Hill, B. Scott, B. Daugherty and Y. Koh, A new predictive equation for resting energy expenditure in healthy individuals, The American Journal of Clinical Nutrition, vol. 51, no. 2, pp. 241-247, 1990.

## 19.2. Project documents

[12] Project repository README.md and ml/README.md, Fitness Habit Analysis and Personalized Workout Recommendation System.

## 19.3. Note on information that still has to be filled in

The following items could not be derived from the project files and are marked in square brackets in this report. They have to be completed by the student before submission:

• College or university name, department and logo on the cover page.

• Student name, guide name, roll number and academic year on the cover page.

• Processor, RAM and storage of the machine used in the hardware requirements table.

• The repository contains no Jupyter notebook; if a notebook is required by the college, it has to be prepared from the modules of the ml package.

• Declaration, certificate and acknowledgement pages, if the college format demands them.